M^{2}RC-EVAL: MASSIVELY MULTILINGUAL REPOSITORY-LEVEL CODE COMPLETION EVALUATION

Jiaheng Liu[1*,†], **Ken Deng**[1*], **Congnan Liu**[1], **Jian Yang**[1], **Shukai Liu**[1], **He Zhu**[1], **Peng Zhao**[1], **Linzheng Chai**[1], **Yanan Wu**[1], **Ke Jin**[1], **Ge Zhang**[2], **Zekun Wang**[2], **Guoan Zhang**[1], **Bangyu Xiang**[1], **Wenbo Su**[1], **Bo Zheng**[1]
[1]Alibaba Group, [2]University of Waterloo
{ljh411989, dengken.deng}@alibaba-inc.com

arXiv:2410.21157v1 [cs.CL] 28 Oct 2024

ABSTRACT

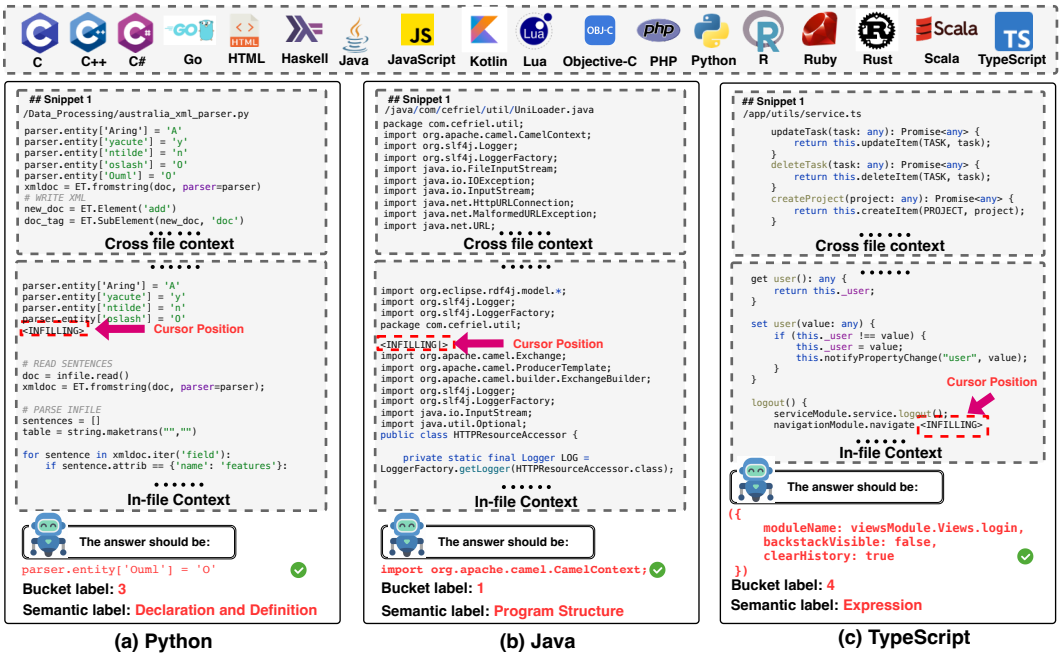
Repository-level code completion has drawn great attention in software engineering, and several benchmark datasets have been introduced. However, existing repository-level code completion benchmarks usually focus on a limited number of languages (<5), which cannot evaluate the general code intelligence abilities across different languages for existing code Large Language Models (LLMs). Besides, the existing benchmarks usually report overall average scores of different languages, where the fine-grained abilities in different completion scenarios are ignored. Therefore, to facilitate the research of code LLMs in multilingual scenarios, we propose a massively multilingual repository-level code completion benchmark covering 18 programming languages (called **M^{2}RC-EVAL**), and two types of fine-grained annotations (i.e., **bucket-level** and **semantic-level**) on different completion scenarios are provided, where we obtain these annotations based on the parsed abstract syntax tree. Moreover, we also curate a massively multilingual instruction corpora **M^{2}RC-INSTRUCT** dataset to improve the repository-level code completion abilities of existing code LLMs. Comprehensive experimental results demonstrate the effectiveness of our M^{2}RC-EVAL and M^{2}RC-INSTRUCT. Code and data will be released at `https://github.com/M2RC-Eval-Team/M2RC-Eval`.

1 INTRODUCTION

The emergence of Large Language Models (LLMs) specifically designed for code-related tasks has marked a significant advancement in code generation. The code LLMs (Roziere et al., 2023; Zheng et al., 2023; Guo et al., 2024a; Hui et al., 2024) pre-trained on extensive datasets comprising billions of code-related tokens further revolutionize the automation of software development tasks, providing contextually relevant code suggestions and facilitating the translation from natural language to code. The generation capability of code LLMs opens up diverse applications in software development, promising to enhance productivity and streamline coding processes. As the field continues to evolve, it presents exciting opportunities for future developments and innovations in automated programming and code assistance.

The code completion task is crucial in modern software development, enhancing coding efficiency and accuracy by predicting and suggesting code segments based on context. Recent advancements in code LLMs (Bavarian et al., 2022b) have introduced sophisticated completion techniques, such as prefix-suffix-middle (PSM) and suffix-prefix-middle (SPM) paradigms, which can complete middle code segments given the surrounding context. However, the current benchmark (Ding et al., 2024; Liu et al., 2023a) mainly focuses on several programming languages. For example, the CrossCodeEval (Ding et al., 2024) includes four languages (i.e., Python, Java, TypeScript, C#). Besides, existing benchmarks can only provide the average score among all samples, which can not provide a language-specific evaluation for different programming languages based on their intrinsic structure.

* First two authors contributed equally. † Corresponding Author: Jiaheng Liu.



(a) Python (b) Java (c) TypeScript

Figure 1: Overview of our proposed M^{2}RC-EVAL with 18 languages. Specifically, first, we provide three samples from different languages (i.e., Python, Java, TypeScript) for illustration, where the bucket label and semantic label for the corresponding cursor position are provided. Second, the code LLMs need to predict the completion results given the in-file context from the current code file and the cross file context retrieved from other code files in the current repository. Note that "$<$ INFILLING $>$" denotes that the current position will be triggered for code completion.

Inspired by the multilingual in-file code generation benchmark MultiPL-E Cassano et al. (2022) and McEval (Chai et al., 2024), we create a massively multilingual repository-level code completion Evaluation benchmark called **M^{2}RC-EVAL** *to facilitate the research of the community.*

In this paper, as shown in Fig. 1, our M^{2}RC-EVAL includes 18 programming languages with two types of fine-grained annotations (i.e., **bucket-level** and **semantic-level**), where each language contains 100 validation and 500 test samples, respectively. Specifically, for the bucket-level annotations, we first generate abstract syntax tree with N layers using code parser (i.e., Treesitter [1]), and divide these N into fixed M buckets, Then, for each completion cursor position, we annotate the corresponding bucket-level label based on the layer to which the location belongs. In this way, we can obtain different code completion scenarios with different difficulties.

For the semantic-level annotations, inspired by (Takerngsaksiri et al., 2024), we first pre-define 11 major semantic labels (e.g., Program Structure, Statement) for each completion cursor position, which aims to analyze the fine-grained performance across different code semantics. Note that as different languages usually have specific syntax, we carefully design the subcategories under each major semantic label for different languages. Then, as the code parser usually provides syntax labels (e.g., functions, variables, classes, empty lines)[2] for each completion cursor position, we carefully define the mappings between the syntax labels to our designed semantic labels and build the semantic-level annotations for our M^{2}RC-EVAL. Finally, to enhance the performance of repository-level code completion for existing code LLMs, we also create a massively multilingual instruction corpora **M^{2}RC-INSTRUCT** of 18 languages.

The contributions are summarized as follows:

- We propose the first massively multilingual repository-level code completion benchmark M^{2}RC-EVAL covering 18 languages, where two types of annotations (bucket-level and semantic-level labels) are provided based on the parsed abstract syntax tree.

[1] `https://tree-sitter.github.io/tree-sitter/`

[2] Note that the syntax label provided by code parser (e.g., tree-sitter) are highly detailed.

- We introduce M^{2}RC-INSTRUCT, the massively multilingual repository-level code instruction corpora covering the multilingual code snippet from 18 languages, which can greatly enhance the performance of repository-level code completion results.
- Comprehensive evaluation results and analysis demonstrate the effectiveness of our proposed M^{2}RC-EVAL and M^{2}RC-INSTRUCT.

2 RELATED WORKS

Code Large Language Models. Code large language models (LLMs) (Chen et al., 2021; Zhao et al., 2024; Black et al., 2021; 2022; Le et al., 2022; Chowdhery et al., 2023; Nijkamp et al., 2023; Fried et al., 2023; Xu et al., 2022) are increasingly involved in modern programming, due to excellent capabilities of code generation (Li et al., 2022; Allal et al., 2023), code repair (Wang et al., 2021; 2023), code translation (Zheng et al., 2023; Li et al., 2023), and other coding tasks. UniCoder (Sun et al., 2024) and SPT-Code (Niu et al., 2022) introduce the pseudo-code generation and the alignment between Abstract Syntax Tree (AST) and code. Recent code LLMs such as Code Llama (Roziere et al., 2023), DeepSeek-Coder (Guo et al., 2024a), and Qwen2.5-Coder (Hui et al., 2024) incorporate the fill-in-the-middle (FIM) task into their training stage for code completion. Moreover, there is a wide variety of in-file benchmarks to evaluate different capabilities of code LLMs (Zheng et al., 2023; Austin et al., 2021; Jain et al., 2024), which focus on a limited range of programming languages (e.g. Python and Java). The recent work (Chai et al., 2024) extends the number of programming languages to 40 for multilingual evaluation scenarios, which has not considered the repository-level code completion.

Repository-level Code Completion. The latest repository-level code completion methods (Bairi et al., 2023; Phan et al., 2024; Liao et al., 2023; Shrivastava et al., 2023a; Agrawal et al., 2023; Shrivastava et al., 2023b; Pei et al., 2023; Zhang et al., 2023) are similar to RAG, aim to precisely retrieve all related code snippets across files within a repository. Further, repository-level benchmarks are proposed to estimate the capability of code LLMs in a more realistic software engineering scenario. But these datasets (Ding et al., 2023; 2022; Allal et al., 2023) are primarily concentrated on several programming languages. Regarding difficulty categorization, most methods only consider the number of files involved in the completion content, overlooking the code's structural and semantic context within the entire project. Repofusion (Shrivastava et al., 2023a) and Repocoder (Zhang et al., 2023) predict one line based on the prefix and suffix code, while CoderEval (Yu et al., 2024) measures how many third-party libraries are called. To comprehensively evaluate the multilingual repository-based code completion of different code LLMs, we push the boundaries of programming languages into 18 languages in M^{2}RC-EVAL with fine-grained annotations.

3 M^{2}RC-EVAL

3.1 DATA COLLECTION

The Overall Data Pool. We begin by collecting The Stack v2 (Lozhkov et al., 2024), which consists of permissively licensed repositories from GitHub. Next, we adopt the `The-stack-v2-dedup`, which includes 784 million source code files spanning 619 programming languages with manual and heuristic pre-processing. Further, we keep only repositories receiving more than 5 stars and containing $[10, 50]$ files. Lastly, preserving files written in 18 common languages, we have 431,353,244 files remaining, constituting the overall data pool.

Completion Cursor Position Selection. Completion cursor position selection significantly impacts the quality of a code completion benchmark. Previous studies (Ding et al., 2024; Liu et al., 2023a) randomly select a segment of consecutive characters as the completion span, which does not guarantee the integrity of identifiers and statements. On the contrary, in M^{2}RC-EVAL, we first parse the abstract syntax tree (AST) of each source code file, and then we randomly choose a node (e.g., the node of "Function Definition" in Fig. 2) on the AST as the completion cursor position. After that, we obtain the corresponding code to obtain the ground-truth for the current completion cursor position. Finally, at inference, the code LLMs need to predict the current code span given the in-file and cross file contexts. Similarly, in training, we just use the ground-truth to supervise the tuning process of the code LLMs.

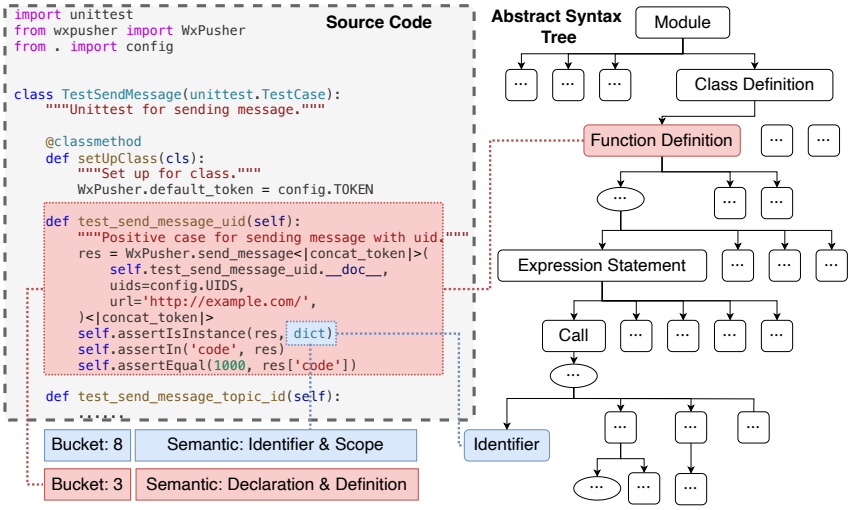


Figure 2: Illustration on generating completion cursor position and fine-grained annotations. Specifically, we first parse the source code into an abstract syntax tree (AST). Then, we choose one node as the completion cursor position and generate the bucket label based on the belonged layer number in AST, and obtain the semantic label based on the node type parsed by the Tree-sitter.

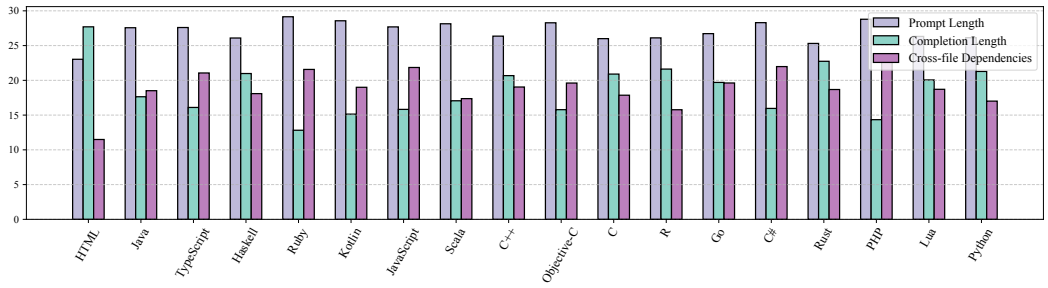


Figure 3: The average prompt length (100x tokens), completion span length (50x tokens), and cross-file dependencies (1x) in the testing set of M^{2}RC-EVAL. We define the number of other files, which are explicitly imported and implicitly referenced by the current file, as cross-file dependencies.

Table 1: A comparison with existing notable repository-level code completion datasets.

Benchmark	# Languages	Fine-grained	Training Set	# Test Repos
RepoBench (Liu et al., 2023a)	2	✗	✓	1669
CrossCodeEval (Ding et al., 2024)	4	✗	✗	1002
R^{2}C^2-Bench (Deng et al., 2024)	4	✗	✓	1353
M^{2}RC-EVAL& M^{2}RC-INSTRUCT	18	✓	✓	5993

3.2 QUALITY CONTROL

We build a suite of post-processing filters to enhance the quality of M^{2}RC-INSTRUCT. We eliminate examples based on two heuristic rules: (1) The completion cursor position should be no longer than 5 lines. (2) If the completion ground truth is fewer than 20 characters, at least 20% of them should be alphabetic. To improve data independence and inference difficulty, we apply extra filters to the test cases in M^{2}RC-EVAL. (a) Repositories in M^{2}RC-EVAL should be absent from M^{2}RC-INSTRUCT. (b) We ensure that 30% of the completion ground truth is not shorter than 2 lines. (c) The completion cursor position should not be fully white-spaced. (d) We discard test cases that could be exactly predicted by `DeepSeekCoder-1.3B` (Guo et al., 2024b) without cross file contexts.

3.3 DATASET STATISTICS

Following the quality filters in §(3.2) from the overall data pool §(3.1). We sample 50,000 files per language to construct our M^{2}RC-INSTRUCT, and sample 100, and 500 files per language to build the validation and test sets of our M^{2}RC-EVAL, respectively. The statistics of the test set are shown in Fig. 3, and we also provide a detailed comparison between our M^{2}RC-EVAL with existing repository-level code completion datasets in Table 1.

3.4 FINE-GRAINED ANNOTATIONS

As shown in Fig. 2, to analyze the performance in a fine-grained manner, we further provide two types of fine-grained annotations (i.e., bucket-level and semantic-level) for each completion cursor. Specifically, we first generate the abstract syntax tree. For the bucket-level annotations, we first simply divide each tree into M buckets based on the depth degree of the abstract syntax tree. Note that we set M as 10 in our M^{2}RC-EVAL. For example, if the number of layers for the current abstract syntax tree is N, the i-th layer of the tree belongs to the $\lceil \frac{i}{N/M} \rceil$ bucket. Then, for each completion cursor node, we annotate the bucket label based on the layer number of each node. Similarly, for the semantic-level annotations, we directly annotate the semantic-level label for each completion cursor node. Specifically, we pre-define 11 major classes (i.e., "Program Structure", "Declaration and Definition", "Control Flow Structure", "Expression", "Data Type", "Statement", "Modifier and Attribute", "Comments and Documentation", "Preprocessing Directive", "Identifier and Scope", "Special Language Structure"). Then, as different languages have many specific designs, the sub-categories under each major class are carefully annotated for different languages. As shown in Fig. 4, we provide the semantic-level annotations on three main-stream programming languages (Java, Go, Scala), where the annotations on the remained 15 languages are provided in Fig. 14 of the Appendix.

(a) Java (b) Go (c) Scala

Figure 4: Semantic-level annotations on different types of programming languages.

4 EXPERIMENTS

We perform detailed experiments on M^{2}RC-EVAL with three popular Code LLMs (i.e., **StarCoder-7B** (Li et al., 2023), **DeepSeekCoder-6.7B** (Guo et al., 2024b) and **Code Llama-7B** (Roziere et al., 2023)) (See Appendix A.3 for more details).

4.1 EVALUATION METRICS

Following (Ding et al., 2023), we compare the generated code with the reference and compute the exact match (**EM**) and edit similarity (**ES**) metrics, which assesses the textural similarities and ignores semantic structure similarities among predictions and ground-truth.

Table 2: Exact match (%) and edit similarity (%) performance on M^{2}RC-EVAL.

Model	**C**		**C#**		**C++**		**Go**		**HTML**		**Haskell**		-	
	EM	ES	EM	ES	EM	ES	EM	ES	EM	ES	EM	ES	EM	ES
Code Llama-7B	18.6	47.2	19.6	52.6	21.8	51.1	26.0	53.6	20.6	40.4	22.6	48.9	-	-
+ Retrieval	21.8	47.2	22.9	48.9	23.2	46.6	23.8	52.4	12.6	35.6	22.6	48.9	-	-
+ Retrieval & Tuning	45.4	72.0	43.5	72.3	50.8	74.9	43.4	72.9	41.8	63.6	39.8	66.3	-	-
StarCoder-7B	20.0	50.4	20.0	53.3	22.4	51.8	25.4	58.2	17.4	40.7	25.0	51.1	-	-
+ Retrieval	23.8	47.8	27.1	53.2	24.6	48.0	26.0	53.6	20.6	40.4	25.0	47.7	-	-
+ Retrieval & Tuning	47.0	72.7	45.1	74.8	52.4	76.3	43.2	73.7	45.8	67.1	44.8	70.2	-	-
DeepSeekCoder-6.7B	22.4	53.7	21.4	56.2	23.2	54.2	29.4	61.4	17.6	43.4	25.2	51.3	-	-
+ Retrieval	28.2	52.6	25.3	52.6	27.6	52.2	29.4	61.4	17.6	43.4	25.8	51.0	-	-
+ Retrieval & Tuning	48.6	75.2	47.9	76.9	54.4	78.2	48.8	78.4	45.0	66.3	45.8	72.0	-	-
Model	**Java**		**JavaScript**		**Kotlin**		**Lua**		**Objective-C**		**PHP**		-	
Code Llama-7B	23.4	58.5	17.2	52.0	23.6	57.0	20.0	45.7	17.8	49.5	19.2	54.9	-	-
+ Retrieval	23.4	57.5	19.6	48.0	20.8	50.0	19.6	42.2	21.4	46.6	21.2	49.0	-	-
+ Retrieval & Tuning	41.8	74.1	38.8	70.1	45.0	75.6	43.8	70.5	49.8	75.9	45.6	76.7	-	-
StarCoder-7B	24.0	59.2	16.6	52.0	24.4	59.3	21.4	48.6	17.6	49.6	18.6	54.4	-	-
+ Retrieval	25.0	53.1	22.0	50.8	22.8	52.6	26.4	48.5	23.6	48.0	18.6	54.4	-	-
+ Retrieval & Tuning	47.4	76.9	38.8	70.1	45.0	75.6	43.8	70.5	50.8	75.9	45.6	76.7	-	-
DeepSeekCoder-6.7B	22.2	61.0	20.4	56.5	26.0	61.0	22.0	48.8	21.0	55.6	24.2	58.6	-	-
+ Retrieval	21.6	51.4	24.4	53.6	26.0	61.0	22.0	49.9	27.6	53.5	28.6	56.9	-	-
+ Retrieval & Tuning	48.2	79.1	43.6	73.5	46.0	75.7	44.6	70.6	52.2	77.6	49.8	78.8	-	-
Model	**Python**		**R**		**Ruby**		**Rust**		**Scala**		**TypeScript**		**Avg.**	
Code Llama-7B	24.6	54.2	15.2	41.2	17.2	45.8	26.2	56.0	22.8	48.5	23.4	52.3	19.4	50.3
+ Retrieval	17.4	46.4	15.2	39.8	17.2	42.3	26.0	51.3	22.8	48.5	19.4	48.6	20.2	46.1
+ Retrieval & Tuning	39.2	69.9	38.6	65.5	43.0	68.5	42.0	69.2	41.0	70.1	37.0	68.2	41.9	70.0
StarCoder-7B	19.4	52.9	16.4	43.7	19.4	47.4	26.2	56.0	23.6	53.4	19.8	53.3	21.0	52.0
+ Retrieval	24.6	54.2	22.6	47.2	23.6	47.4	26.4	53.5	22.8	48.5	23.4	52.3	24.1	50.0
+ Retrieval & Tuning	39.2	69.9	41.0	66.6	43.0	68.5	45.8	72.6	43.6	71.5	39.2	69.7	44.5	72.2
DeepSeekCoder-6.7B	21.8	55.1	19.4	48.5	23.6	52.2	23.8	54.3	24.6	56.7	19.4	55.4	22.6	54.7
+ Retrieval	21.8	55.1	19.4	48.5	23.6	52.2	23.8	54.3	22.4	50.4	26.0	54.5	25.1	51.7
+ Retrieval & Tuning	41.6	71.3	45.4	69.4	45.6	70.3	47.6	73.4	44.8	73.7	43.2	73.4	**46.8**	**74.1**

4.2 EXPERIMENTAL SETUP

Baseline. Only the original code file, where the cursor position is located, is provided for the code LLMs. As no explicit inter-file context is supplied, the model must utilize its inherent knowledge-based reasoning abilities to generate appropriate code.

+ Retrieval. In line with the approach outlined in CrossCodeEval (Ding et al., 2023), the retrieval process begins by examining files within the same repository. Continuous code segments of L lines are extracted, where L matches the length of the retrieval query and is set as 10 by default. Subsequently, these extracted candidates are prioritized based on their Jaccard similarity scores. The most relevant fragments are then appended to the beginning of the in-file context in descending order of similarity. This concatenation continues until the total length, including both the added candidates and the original in-file context, reaches the predetermined maximum token limit of 4096.

+ Retrieval & Tuning. To further improve the performance of repository-level code completion, we fine-tune code LLMs on the M^{2}RC-INSTRUCT dataset mentioned in §(3). At inference, we use the same inference strategy as discussed in "+ Retrieval".

4.3 MAIN RESULTS

We present the results on M^{2}RC-EVAL in Table 2. We observe that different code LLMs have different repository-level code completion abilities for different programming languages. For instance, DeepSeekCoder-6.7B demonstrates strong completion ability for Go, while its performance is weaker with HTML, a markup language, which demonstrates the necessity of evaluating code LLMs for multilingual capabilities. Besides, the results indicate that cross file context is highly effective, resulting in a significant improvement compared to using only in-file context. In particular, the multilingual SFT on our created instruction corpora M^{2}RC-INSTRUCT also significantly

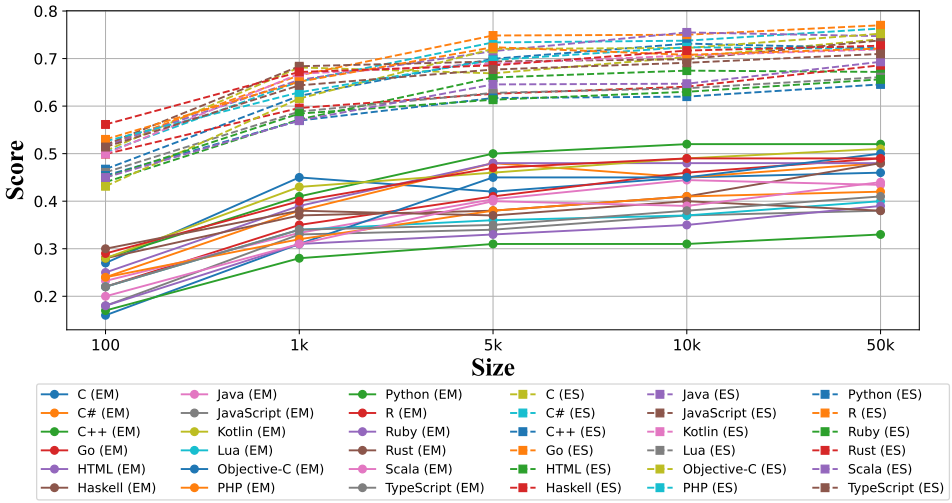


Figure 5: Effectiveness of using different training data sizes.

enhances performance on M^{2}RC-EVAL. Notably, after SFT on M^{2}RC-INSTRUCT, Code Llama-7B, which originally ranked lowest with in-file context, outperformed the non-finetuned StarCoder-7B, demonstrating the effectiveness of M^{2}RC-INSTRUCT.

4.4 ANALYSIS

Analysis on different model sizes. In Table 3, we provide the performance of StarCoder with respect to different model sizes in the validation set of M^{2}RC-EVAL. Notably, StarCoder-7B consistently outperforms StarCoder-3B under comparable conditions. However, following the application of SFT on M^{2}RC-INSTRUCT, the results of StarCoder-3B exceed those of the inference-only StarCoder-7B. This finding underscores the effectiveness of our M^{2}RC-INSTRUCT in augmenting the capabilities of smaller models in repository-level code completion.

Analysis on different training data sizes. In Fig. 5, we conduct a evaluation of the fine-tuned StarCoder-7B by employing varying sizes of M^{2}RC-INSTRUCT and report the results on validation set M^{2}RC-EVAL. Our observations indicate that increasing the dataset from 0.1k to 50k samples per language yields improved results. This suggests that more training data can help boost the model's performance. Therefore, we select 50k samples per language as the default training set size for our M^{2}RC-EVAL.

Table 3: Performance on M^{2}RC-EVAL.

Model	Average	
	EM	ES
StarCoder-3B	14.9	43.5
+ Retrieval	14.6	38.4
+ Retrieval & Tuning	41.7	69.1
StarCoder-7B	20.6	49.9
+ Retrieval	23.6	49.3
+ Retrieval & Tuning	44.4	71.4

Analysis on the granularity of different bucket levels. As mentioned in §(3.4), we categorize M^{2}RC-EVAL into ten bucket levels based on the positions of the code requiring completion within the abstract syntax tree. As shown in Fig. 6, we presents the performance of StarCoder-7B on the test set of M^{2}RC-EVAL across these different bucket levels, and we observe that as the bucket level decreases, the performance of StarCoder-7B correspondingly declines, which means that the code completion on the shadow layer is usually more challenging than on the deep layer. For more experimental data on single-language completion performance and its relation to bucket levels, please refer to Fig.11 and Fig.12 in the Appendix. These findings suggest that the code LLMs encounter challenges when addressing shallow nodes within the syntax tree during the code completion process.

Analysis on the granularity of different semantic levels. Similarly, in §(3.4), we also categorize the nodes within the abstract syntax tree into eleven primary semantic levels based on their

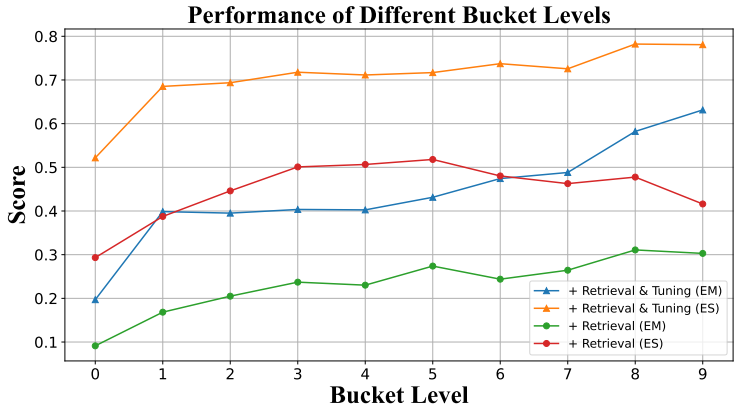


Figure 6: Effectiveness of different bucket levels based on StarCoder-7B.

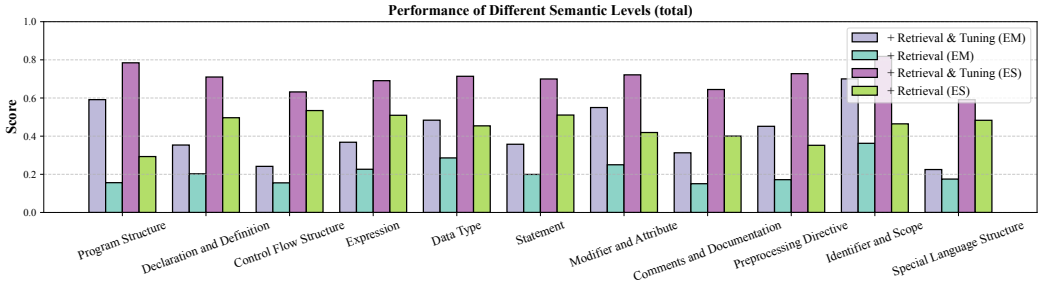


Figure 7: Effectiveness of different semantic levels based on StarCoder-7B.

semantic characteristics, and we provide the performance of StarCoder-7B on repository-level code completion for these various semantic levels across multilingual languages on the test set of the M^{2}RC-EVAL. Notably, we observe significant performance disparities across different semantic levels. Specifically, StarCoder-7B shows superior performance on "Identifier and Scope", while it exhibits lower efficacy on "Special Language Structure", This suggests that current code LLMs are proficient at completing tasks related to variable definitions and references, yet their capacity to handle characteristics of different languages requires further enhancement. For single-language completion performance across various node types, please refer to Fig. 13 in the Appendix.

Analysis on completion on different lines. To explore the differences in the code completion capabilities of code LLMs for varying line counts, we conduct a statistical analysis of the performance of StarCoder-7B ("Retrieval & Tuning") across various completion lengths on the test set of M^{2}RC-EVAL. As shown in Fig.8, StarCoder-7B can effectively complete tasks involving a small number of lines. However, as the number of lines to be completed increases, the scores of the generated code gradually declines. This indicates that completing multi-line code remains a challenge for code LLMs. For better illustration, we also provide the results of different difficulties for different languages in Fig. 9, where we define completion on 1 line, completion on 2-3 lines and completion on 4-5 lines as easy, middle, and hard settings, respectively.

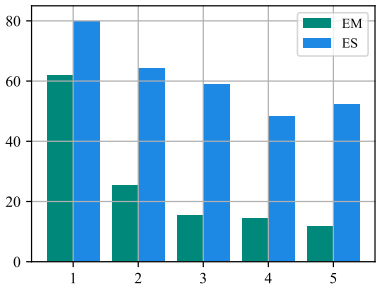


Figure 8: Effectiveness of code completion on different lines based on StarCoder-7B.

Analysis on cross-lingual transfer. We fine-tune the StarCoder-7B model using Python-only data (50k) in M^{2}RC-INSTRUCT and compare it with the results of using our whole training data. In Table 5, we report the results on the validation set of M^{2}RC-EVAL, and observe that fine-tuning the model exclusively with Python data resulted in a significant improvement in its M^{2}RC-EVAL score, coming close to the ES performance achieved through fine-tuning with data from 18 languages. This outcome suggests that the StarCoder-7B base model already demonstrates strong coding proficiency, but lacks robust instruction-following capabilities. Thus, even when fine-tuned solely on Python data, the model is

Table 4: CodeBLEU results on ten representative programming languages.

Model	C	C#	C++	Go	Java	JavaScript	PHP	Python	Ruby	Rust	Avg.
StarCoder-7B	48.3	48.9	50.4	51.5	50.6	46.4	48.2	46.4	46.1	50.4	48.7
+ Retrieval	50.1	52.3	51.1	52.5	51.4	49.3	52.2	49.3	49.1	51.4	50.9
+ Retrieval & Tuning	56.0	57.4	57.6	57.0	57.6	54.8	57.8	52.0	52.9	55.5	**55.9**

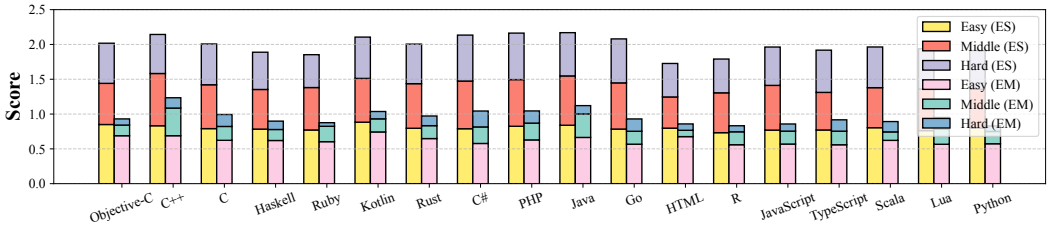


Figure 9: Performance on M^{2}RC-EVAL for problems of different difficulty levels.

able to effectively transfer instruction-following skills to other languages, thereby achieving exceptional multilingual performance.

Analysis on CodeBLEU metric. In Table 2, we mainly report the EM and ES metrics based on the textural similarity, which neglects important syntactic and semantic features of codes and underestimates different outputs with the same semantic logic. Thus, the CodeBLEU (Ren et al., 2020) [3] is proposed, which considers information from not only the shallow match, but also the syntactic match and the semantic match. In Table 4, we report the results of 10 popular programming languages using the test split of M^{2}RC-EVAL based on the StarCoder-7B model and observe that we can still achieve better performance by fine-tuning on our constructed M^{2}RC-INSTRUCT, which further demonstrates the effectiveness of our M^{2}RC-INSTRUCT on repository-level code completion.

Table 5: Performance on M^{2}RC-EVAL.

Model	Average	
	EM	ES
+ Retrieval	23.6	49.3
+ Retrieval & Tuning	44.4	71.4
+ Retrieval & Tuning (Python Only)	39.2	67.9

Analysis on various input lengths. As shown in Fig. 10, we report the results produced by StarCoder-7B ("Retrieval & Tuning") on our M^{2}RC-EVAL when the input lengths of range in {512, 1024, 2048, 4096} tokens. In Fig. 10, we observe that a scaling law exists, where better performance is achieved when the input length is larger. Thus, we set the default input length as 4096 tokens.

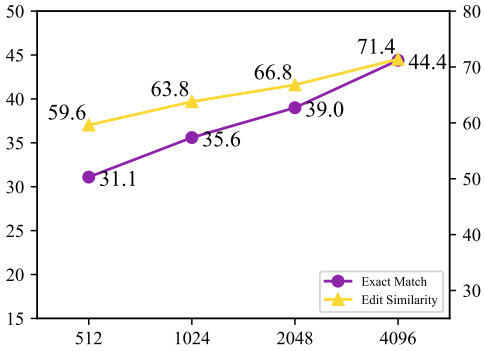


Figure 10: Performance on M^{2}RC-EVAL with various input lengths based on StarCoder-7B.

5 CONCLUSION

In this paper, we propose the first massively multilingual repository-level code completion benchmark (M^{2}RC-EVAL) with 18 popular programming languages, where two types of fine-grained annotations (bucket-level and semantic-level) are provided to comprehensively analyze the effectiveness of different code LLMs. Besides, we also curate a high-quality instruction corpus M^{2}RC-INSTRUCT to enhance the performance of existing models on repository-level code completion. Extensive experi-

[3] We test the CodeBLEU metric based on `https://github.com/k4black/codebleu`.

mental results and detailed discussions demonstrate the effectiveness of our proposed M^{2}RC-EVAL and M^{2}RC-INSTRUCT. Finally, we hope M^{2}RC-EVAL could guide the developers and researchers to understand the repository-level code completion capabilities of LLMs and facilitate the growth of code intelligence and software engineering.

REFERENCES

Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K Lahiri, and Sriram K Rajamani. Guiding language models of code with global context using monitors. 2023.

Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, et al. Santacoder: don't reach for the stars! *arXiv preprint arXiv:2301.03988*, 2023. URL https://arxiv.org/abs/2301.03988.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL https://arxiv.org/abs/2108.07732.

Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B Ashok, Shashank Shet, et al. Codeplan: Repository-level coding using llms and planning. *arXiv preprint arXiv:2309.12499*, 2023.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022a. URL https://arxiv.org/abs/2207.14255.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022b.

Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, 2021. URL https://doi.org/10.5281/zenodo.5297715.

Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, Usvsn Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*, pp. 95–136, virtual+Dublin, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.bigscience-1.9. URL https://aclanthology.org/2022.bigscience-1.9.

Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. Multiple: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*, 2022.

Linzheng Chai, Shukai Liu, Jian Yang, Yuwei Yin, Ke Jin, Jiaheng Liu, Tao Sun, Ge Zhang, Changyu Ren, Hongcheng Guo, et al. Mceval: Massively multilingual code evaluation. *arXiv preprint arXiv:2406.07436*, 2024.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *ArXiv preprint*, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. 24(240):1–113, 2023.

CodeGeeX, 2022. https://github.com/THUDM/CodeGeeX.

Ken Deng, Jiaheng Liu, He Zhu, Congnan Liu, Jingxin Li, Jiakai Wang, Peng Zhao, Chenchen Zhang, Yanan Wu, Xueqiao Yin, Yuanxing Zhang, Wenbo Su, Bangyu Xiang, Tiezheng Ge, and Bo Zheng. R2c2-coder: Enhancing and benchmarking real-world repository-level code completion abilities of code large language models. *ArXiv*, abs/2406.01359, 2024.

Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. Cocomic: Code completion by jointly modeling in-file and cross-file context. *arXiv preprint arXiv:2212.10007*, 2022. URL https://arxiv.org/abs/2212.10007.

Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/920f2dced7d32ab2ba2f1970bc306af6-Abstract-Datasets_and_Benchmarks.html.

Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *Advances in Neural Information Processing Systems*, 36, 2024.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. Incoder: A generative model for code infilling and synthesis. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=hQwb-lbM6EL.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024a. URL https://arxiv.org/abs/2401.14196.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024b.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022. URL https://arxiv.org/abs/2211.15533.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *ArXiv*, abs/2207.01780, 2022. URL https://api.semanticscholar.org/CorpusID:250280117.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023. URL https://arxiv.org/abs/2305.06161.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *ArXiv preprint*, abs/2203.07814, 2022. URL https://arxiv.org/abs/2203.07814.

Dianshu Liao, Shidong Pan, Qing Huang, Xiaoxue Ren, Zhenchang Xing, Huan Jin, and Qinying Li. Context-aware code generation framework for code repositories: Local, global, and third-party library awareness. 2023.

Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023a.

Tianyang Liu, Canwen Xu, and Julian J. McAuley. Repobench: Benchmarking repository-level code auto-completion systems. abs/2306.03091, 2023b. doi: 10.48550/ARXIV.2306.03091. URL https://doi.org/10.48550/arXiv.2306.03091.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. Starcoder 2 and the stack v2: The next generation. 2024.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=iaYcJKpY2B_.

Changan Niu, Chuanyi Li, Vincent Ng, Jidong Ge, LiGuo Huang, and Bin Luo. Spt-code: Sequence-to-sequence pre-training for learning source code representations. *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, pp. 01–13, 2022. URL https://api.semanticscholar.org/CorpusID:246077487.

Hengzhi Pei, Jinman Zhao, Leonard Lausen, Sheng Zha, and George Karypis. Better context makes better code language models: A case study on function call argument completion. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'23/IAAI'23/EAAI'23. AAAI Press, 2023. ISBN 978-1-57735-880-0. doi: 10.1609/aaai.v37i4.25653. URL https://doi.org/10.1609/aaai.v37i4.25653.

Huy Nhat Phan, Hoang N. Phan, Tien N. Nguyen, and Nghi D. Q. Bui. Repohyper: Search-expand-refine on semantic graphs for repository-level code completion. 2024. URL https://api.semanticscholar.org/CorpusID:271860296.

Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*, 2020.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. 2023.

Disha Shrivastava, Denis Kocetkov, Harm de Vries, Dzmitry Bahdanau, and Torsten Scholak. Repofusion: Training code models to understand your repository. *arXiv preprint arXiv:2306.10998*, 2023a.

Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. Repository-level prompt generation for large language models of code. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 31693–31715. PMLR, 23–29 Jul 2023b. URL https://proceedings.mlr.press/v202/shrivastava23a.html.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. 568:127063, 2024.

Tao Sun, Linzheng Chai, Yuwei Yin Jian Yang, Hongcheng Guo, Jiaheng Liu, Bing Wang, Liqun Yang, and Zhoujun Li. Unicoder: Scaling code large language model via universal code. *ACL*, 2024.

Wannita Takerngsaksiri, Chakkrit Tantithamthavorn, and Yuan-Fang Li. Syntax-aware on-the-fly code completion. *Inf. Softw. Technol.*, 165(C), January 2024. ISSN 0950-5849.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. 2023.

Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 8696–8708, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.685. URL https://aclanthology.org/2021.emnlp-main.685.

Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. Codet5+: Open code large language models for code understanding and generation. 2023.

Frank F Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, pp. 1–10, 2022.

Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pp. 1–12, 2024.

Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*, 2023. URL https://arxiv.org/abs/2303.12570.

CodeGemma Team Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A. Choquette-Choo, Jingyue Shen, Joe Kelley, Kshi tij Bansal, Luke Vilnis, Mateo Wirth, Paul Michel, Peter Choy, Pratik Joshi, Ravin Kumar, Sarmad Hashmi, Shubham Agrawal, Zhitao Gong, Jane Fine, Tris Brian Warkentin, Ale Jakse Hartman, Bin Ni, Kathy Korevec, Kelly Schaefer, and Scott Huffman. Codegemma: Open code models based on gemma. *ArXiv*, abs/2406.11409, 2024. URL https://api.semanticscholar.org/CorpusID:270560319.

Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x. *arXiv preprint arXiv:2303.17568*, abs/2303.17568, 2023. doi: 10.48550/ARXIV.2303.17568. URL https://doi.org/10.48550/arXiv.2303.17568.

A APPENDIX

A.1 BROADER IMPACTS

In this paper, we propose a repository-level code completion benchmark with 18 programming languages. Therefore, we hope our work can enhance the improvements on the multilingual repository-level code completion task.

A.2 LIMITATIONS

First, there are several hyperparameters (e.g., training sizes, input length) to tune, which is laborious and expensive. Second, the current work only focuses on the repository-level code completion task, where other repository-level code intelligence tasks are not considered. Third, only textual similarity scores (EM and ES) are used and execution-based evaluation based on test cases is not applied, which may not reflect the performance of different code LLMs well.

A.3 DETAILS OF THE BASELINE MODELS

StarCoder (Li et al., 2023) is a series of generative language models (e.g., 7B, 15.5B). These decoder-only models are trained on the Stack dataset (Kocetkov et al., 2022) and can support 8K tokens in context.

DeepSeekCoder (Guo et al., 2024b) is a collection of code-oriented models with capacities from 1.3B to 33B parameters. Trained on a manually curated 2-trillion-token corpus, these models leverage Fill-in-the-Middle (FIM) (Bavarian et al., 2022a) and Rotary Position Embedding (RoPE) (Su et al., 2024) techniques, which enables efficient code generation and infilling within a 16K token window.

Code Llama (Roziere et al., 2023) is a family of code large language models based on Llama 2 (Touvron et al., 2023) with 7B, 13B, 34B, and 70B parameters. While trained on 16K token sequences, these models can handle inputs up to 100K tokens during inference.

A.4 DISCUSSION ON NO EXECUTION-BASED EVALUATION

Current datasets for repository-level code completion evaluation, such as CrossCodeEval (Ding et al., 2023) and RepoBench (Liu et al., 2023b), only assess textual similarity between predictions and ground-truth. We hypothesize this limitation stems from several challenges: Firstly, generating comprehensive unit tests for each completion position in a repository is problematic. Single-line completions often fail to construct executable functions, and ensuring adequate test coverage is difficult. Secondly, execution-based evaluation necessitates creating diverse environments for each repository, accommodating various software packages and hardware requirements. This process is intricate and challenging to implement. Thirdly, existing benchmarks with unit tests typically focus on simpler scenarios, like single-file completions or function body generation. Examples include commonly used datasets such as Humaneval CodeGeeX (2022) and MBPP Austin et al. (2021). Despite these obstacles, we recognize the importance of execution-based evaluation for accurately assessing code completion effectiveness, and we will continue to investigate how to evaluate repository-level code completion well.

A.5 MORE EXPERIMENTS

- We provide the analysis on the bucket levels in Fig. 11 and Fig. 12, respectively.
- We analyze the effect of different semantic levels on Rust, Objective-C and Haskell in Fig. 13. respectively.
- We provide the semantic-level annotations on another 15 languages in Fig. 14.
- We provide the visualization on the completion cases in Fig. 15 and Fig. 16.

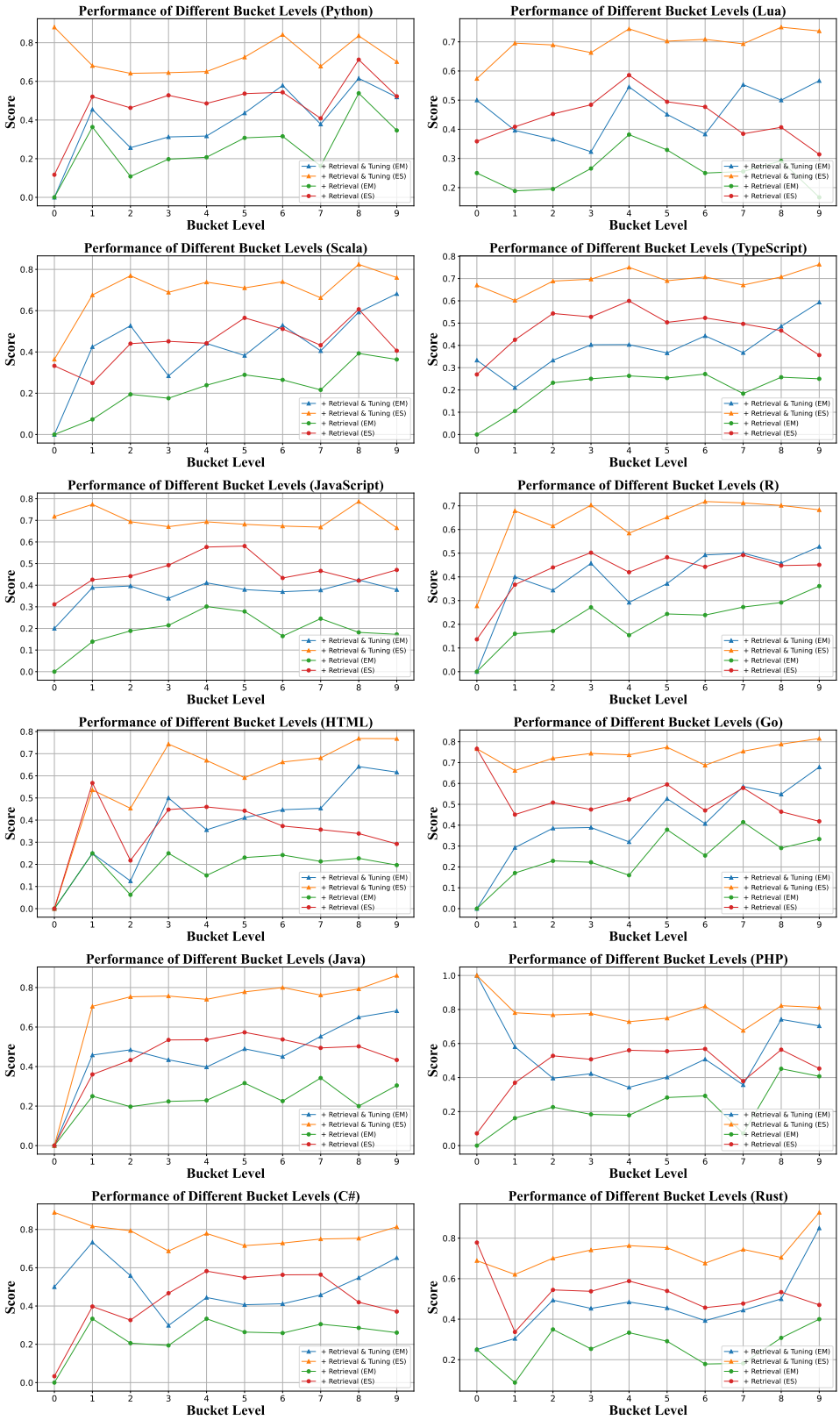


Figure 11: Effectiveness of different bucket levels based on StarCoder-7B for different languages.

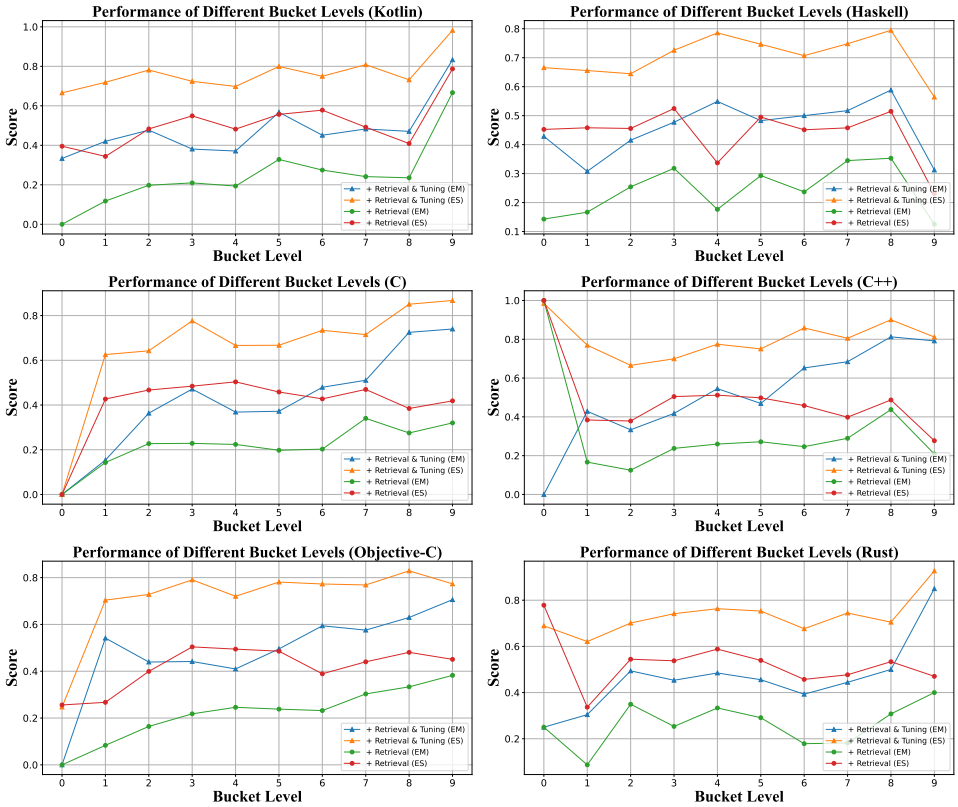


Figure 12: Effectiveness of different bucket levels based on StarCoder-7B for different languages.

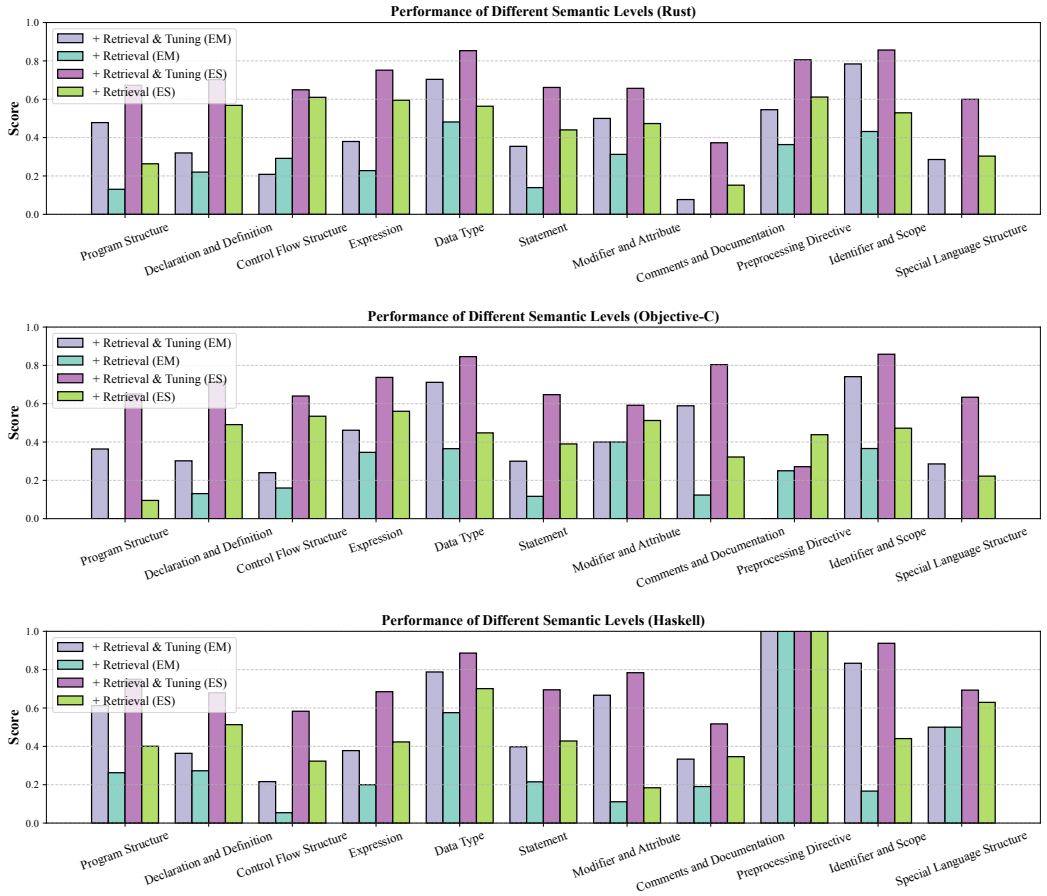
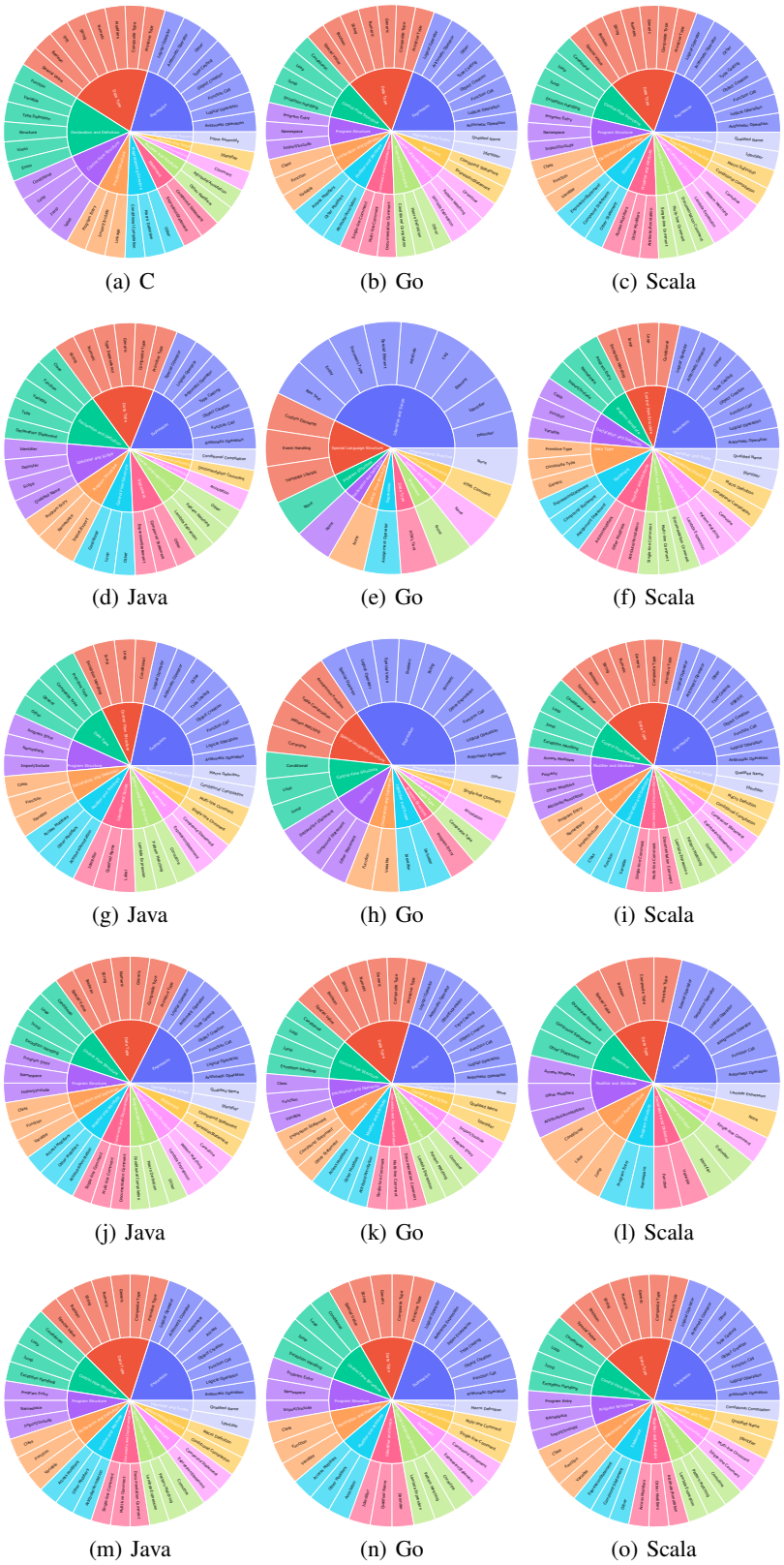


Figure 13: Effectiveness of different semantic levels based on StarCoder-7B.



(a) C (b) Go (c) Scala

(d) Java (e) Go (f) Scala

(g) Java (h) Go (i) Scala

(j) Java (k) Go (l) Scala

(m) Java (n) Go (o) Scala

Figure 14: Semantic-level annotations on different types of programming languages. “none” is used if this language does not have corresponding subcategories.

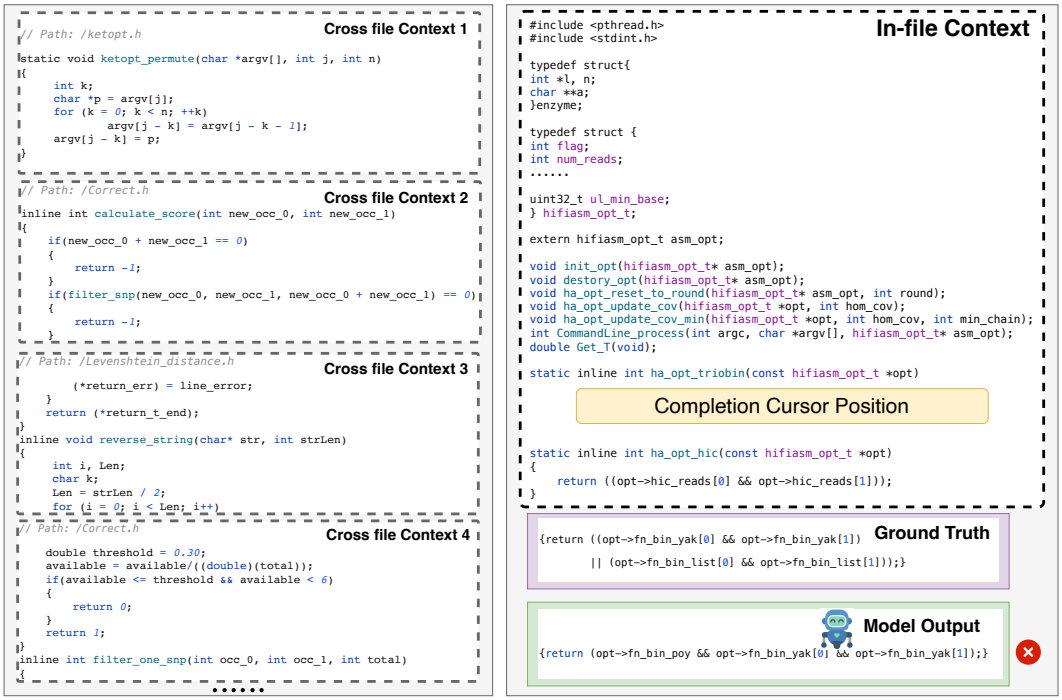


Figure 15: Visualization on failure case for the C language.

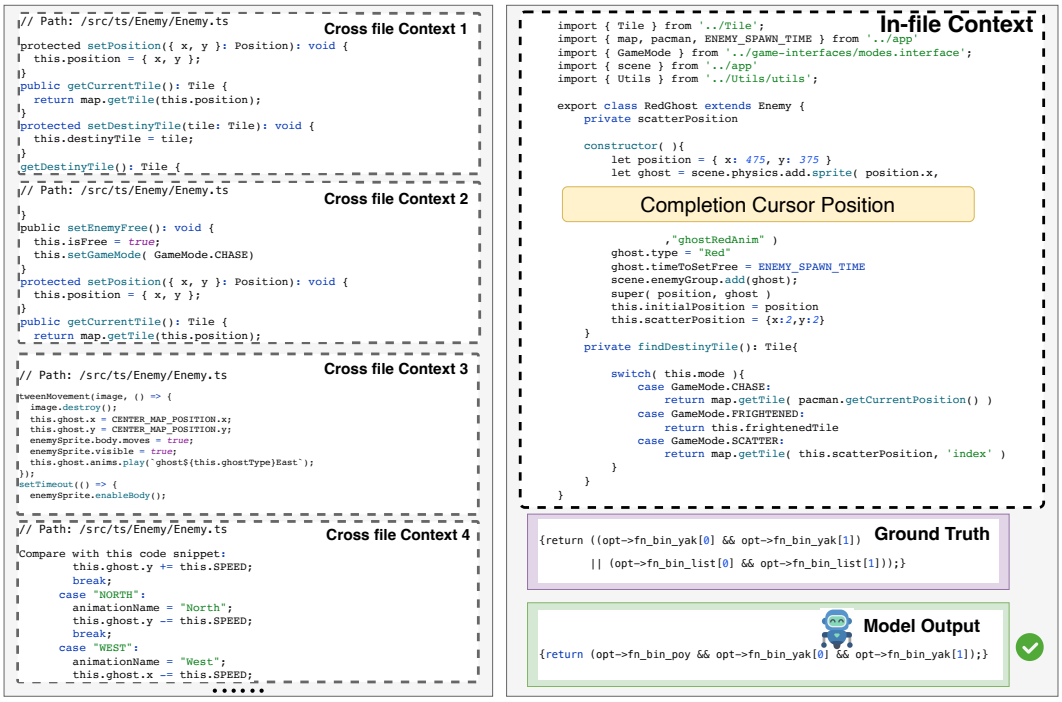


Figure 16: Visualization on success case for the TypeScript language.